A 10,000-star code graph on our bench

15 Oct 2026 · 6 min read
written by Claude from this repository's data

A friend asked a fair question: why build our own code tools when projects like graphify (about 127,000 GitHub stars) and Trail-Code (about 10,000) put a whole codebase into a graph that Claude can query, and report large savings doing it? Someone they knew had cut their Claude usage sharply with one of them.

So we put one on our bench. This note is about what happened, and about how much a benchmark like ours can and cannot say about a tool built for a different job.

Two tools, one idea

Both tools parse a repository with tree-sitter into a graph of definitions and calls, and hand Claude ways to query it. Neither needs a language model to build the code graph, so an existing project gets one right away; that was one of the myths we had going in.

They differ in the thing that turned out to matter most: **keeping the graph current**. graphify's Claude Code setup asks the model, in CLAUDE.md, to run `graphify update .` after changing code, or relies on opt-in git hooks and a file watcher; its issue tracker has reports of answers from graphs days old (#2216, #3718). Trail-Code checks the working tree before every query (a few milliseconds) and rebuilds what changed, with no model involved (`src/graph/refresh.ts`). We benchmarked Trail-Code, the one whose design does not depend on Claude remembering a chore.

What they claim, and what others measured

Trail-Code reports a 162-run sweep with Claude Sonnet 5 answering questions about two repositories: 32% lower cost, 46% fewer tool calls, the same correctness (93% vs 93%), judged by another model (README). It also reports SWE-bench Verified, 27 against 33 resolved out of 50; in the Hacker News discussion a commenter put that difference at p≈0.22, and the maintainer explained the growing sample size by usage limits running out (HN).

graphify's documented benchmark is "71.5x fewer tokens per query" on a 52-file corpus (docs/how-it-works.md). Its own benchmark script compares one query's output against reading **the whole corpus**, and the same page reports about 1x on a 6-file corpus. That is a ceiling, not what an agent that greps and reads a few files would pay.

Two independent checks stand out, both from users in Trail-Code's own issue tracker:

- #117: on 186 cases built from merged pull requests in four repositories (graft 0.10.1), the tool's ranking put 48.5% of the files each pull request changed into its top 10, against **66.5% for plain BM25 keyword search over whole files**. A contributor later proposed adopting whole-file BM25 (#257); the maintainers declined after their own re-test found their current ranking as good or better.
- #506: on a 291-file Swift/Kotlin repository, the context packs it returned were 2.5–4 times larger than `rg` plus a targeted read, and its top hit was right in one of three questions.

Our setup

- Trail-Code 0.22.1 from npm, wired with its own `trail init` (skill, hooks, MCP server, its graph of click: 2,028 nodes and 3,221 edges from 79 files, built in about a second). We kept only the repository-level wiring, so its hooks ran once (#497 describes them running twice when both levels are installed), started its MCP server from the installed version, and turned its telemetry off.
- **Both arms had our pack loaded.** This measures what Trail-Code adds to the pack, not Trail-Code against plain Claude Code. That is the question a pack user has, and it is the harder test for Trail-Code: lsp-plus already answers symbol searches.
- Opus 5.5, click 8.5.0, eight tasks fresh (four runs each) and seven in a long session that had already read the code (two runs each). Every result graded by hidden tests or fixed rules.

What we measured

	Trail-Code added to the pack	95% interval	Runs
Fresh session	**+12.8%** cost	+7.5% to +18.3%	64
Long session	**+32.1%** cost	+21.9% to +43.6%	28
Fresh, without its learning hook	+4.0% cost	+0.1% to +8.0%	64

It cost more on every one of the eight tasks, fresh and long. Quality was the same: every answer correct.

Where the extra cost came from

A learning note that ate the answer. Trail-Code's end-of-turn hook stops Claude once per session to write a "team learning" into the repository. On the explanation task, in all four fresh runs, Claude explained how click resolves an option's value, and then the hook made it write a learning file, so its final message was about the file, not the answer. The note ended with a line for a contributor named from the

repository's history: *"Commit what you learned and their agents get to a PR 3x faster"* (`src/notes/notes.ts`). In an interactive session you would see both messages; in a headless run the final message is the result. We graded Trail-Code fairly on all of Claude's messages (all correct), and measured the hook separately: it accounts for about 8 points of the cost [+4.0%, +13.5%], and 2–4 extra requests on that task. The learning file's name also contained the git user's name.

Context Claude did not need. Without the learning hook, Trail-Code still cost 4% more. Its hooks put an orientation into the session and pointers into prompts, and every later request re-reads them. In the long session, where Claude had already read every file, that added context repeated what it knew, and the cost difference grew to 32%.

Tools Claude rarely reached for. Claude used Trail-Code's command line in 11 of its 32 fresh runs, mostly on the lookup and explanation tasks, and its six MCP tools in none. On the rename, the feature and the tests it worked as without it. That matches what Trail-Code's own maintainer wrote (see note 2), and what we saw with our own hints.

What this does and does not say

Our bench is a 79-file Python library, and our tasks mostly name what they are about: `format_filename`, `ParamType`, an option's default. Opus with the pack finds such things in two or three steps. That is the situation where a pre-built map has least to add, and the situation our users are in most of the time.

It says nothing about the job these tools are designed for: a large, unfamiliar repository where the question comes before anyone knows a symbol to search ("where does authentication happen?"). We did not test that, and the one independent test of that step we found (#117) favoured plain keyword search over the graph's ranking of that version. If you maintain one of these tools and think our setup is unfair to it, the harness is in `benchmarks/` and we will run what you suggest.

We measured it once more on real bug reports, where it reports its own SWE-bench results: 30 issues, again more expensive than the pack alone (+10.8% [0.0%, +25.5%]; note 9). What we took from these tools is in the next note.

Method & limits

Arms: Z (the pack as committed), T (the pack and Trail-Code as `trail init` wires a repository), TG (as T without Trail-Code's Stop hook), PL and TL (Z and T in a prepared 220k-token session). The four explanation runs of T score 0, 0, 0.33 and 0 in the results file, graded on the final message as the bench does; graded on all of Claude's messages, from the transcripts that stay in the lab, all four score 1.0. Costs from transcripts at list prices; intervals are 95% bootstrap intervals over runs within tasks. Four

runs per task fresh, two in the long session: per-task differences are within the runs' spread; the totals' intervals are not. Data: `results-trail.jsonl`; setup: `trail_init()` and `TRAIL_ARMS` in `bench.py`.

Sources

- trailhq, Trail-Code (formerly graft), README, source and issues #117, #257, #497, #506; checked 15 Oct 2026.
- Graphify-Labs, graphify, docs/how-it-works.md, issues #2216, #3718; checked 15 Oct 2026.
- Show HN: graft, discussion of its SWE-bench sample.
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Jimenez, C. E., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR 2024*.
- OpenAI (2024). Introducing SWE-bench Verified.